Checking the Learned Dissipativity Conditions Off the Training Data: Does the All-VSG Kundur Controller Hold Up?

Sehaj Randhir Singh · October 2026 · reproduction of the all-VSG Kundur example in Wang, Wang, Zhuang, Moffat and Dörfler, arXiv:2510.22324

Do the learned dissipativity conditions hold over the region the stability argument uses, and not only at the training tuples? In my reproduction, no. The dissipativity inequality (5b) holds at every one of 98,069 training tuples and fails at 84% of points in a dense sample of the region once u = π(x), which is the input Theorem 2 actually uses. Because this is my rebuild of an example with no public code and several unstated hyperparameters, I read it as a question about how the conditions are sampled during training, and not yet as a statement about the authors' controller.

Why the training set can miss this

The inequality is trained on tuples where u is drawn uniformly at random, and at those tuples the supply rate s(x, u) is large, with a median of about 85 against a storage change of about 0.006 per step, so the inequality is easy to satisfy. The stability argument doesn't consume those tuples. It uses the closed loop, and at $u = \pi(x)$ the supply rate drops to its minimum, $-x^{T}\Phi(x)x$, which is small near the equilibrium. That's the one input where the inequality has the least slack, and a uniform draw over the input range almost never lands on it. So a training loss of zero on random u says very little about the closed-loop case, and the check has to be run there directly.

Method: rebuilding the example

I rebuilt the Kundur two-area system with all four generators as VSGs, using the paper's parameters (virtual impedance j0.3, H = 4 s, damping 5 pu on the 900 MVA base) in a Kron-reduced second-order swing model with a 0.5 ms zero-order-hold step and one B8-B9 circuit open after the fault. Without control it loses synchronism after the fault, as in the paper's Fig. 6, which was the check that the plant was right before training anything. I fixed three model errors on the way there: the post-fault equilibrium solve now uses a slack bus with continuation, speed is no longer double-counted in the step, and inertia is in per-unit speed units, $M = 2H \cdot S/S_{base}$.

The controller follows the paper's parametrization, matrix networks W(x), Q(x), S(x) and $R^{-1}(x)$ with $\pi(x) = -R^{-1}(x)S(x)^{T}x$, trained on the paper's losses for dissipativity (eq. 12), the stability condition $\Phi = S\,R^{-1}\,S^{T} - Q$ (eq. 13), cost shaping and L1, on 2,500 one-second trajectories. Where the paper doesn't fix a choice I made one and recorded it: an input-range penalty so π(x) stays inside the trained u range, which Theorem 2 assumes, mean-plus-max aggregation of the losses in place of a pure batch maximum, and my own epsilons and cost-shaping form.

I then checked four conditions along the closed loop, the inequality (5b), the decrease $V(x^{+}) \le V(x)$, $\Phi(x) \succ 0$ and the input range, with a dense uniform sample of 200,000 points over the paper's region X ($|\Delta\delta| \le 2\pi$, $|\Delta\omega| \le 0.1$ pu), 50,000 points each in boxes at 5% and 1% of X around the equilibrium, and 512 multi-start gradient attacks.

Result

Condition	Training tuples (random u)	Closed loop, whole X	Closed loop, box at 5% of X	Closed loop, box at 1% of X	Worst value found
Dissipativity inequality (5b)	0 of 98,069 violated	84%	100%	100%	−7.13 (dense), −9.60 (attack)
Decrease $V(x^+) \le V(x)$	not trained directly	47%	50%	50%	−0.103 (dense), −0.155 (attack)
$\Phi(x) \succ 0$	trained by hinge	23%	0%	0%	−0.034 (dense)
Input range $\lvert\pi(x)\rvert \le u_{max}$	trained by penalty	0%	0%	0%	holds everywhere sampled

Percentages are shares of the dense sample that violate the condition. "Worst value" is the most negative margin, so negative means violated. All numbers are search results, not certificates.

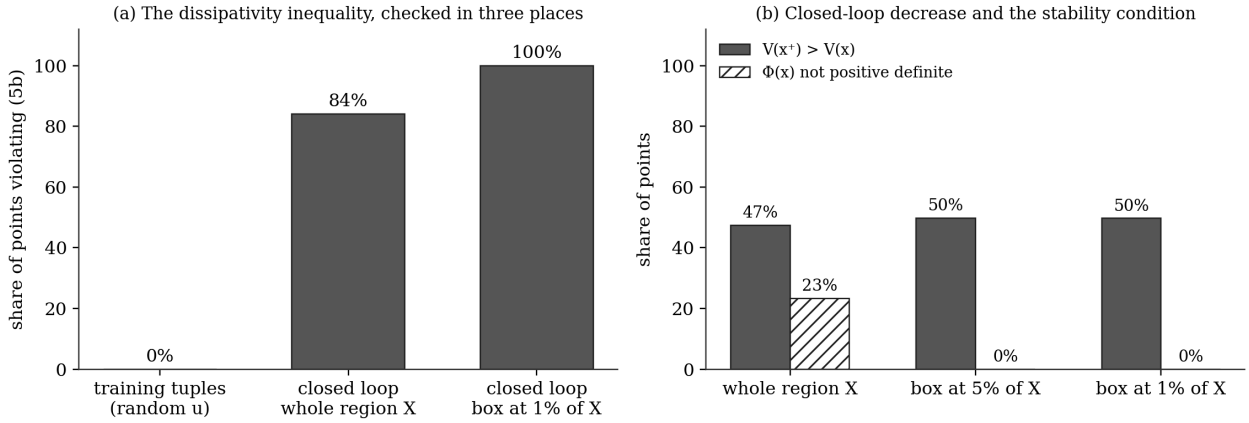


Figure 1. (a) The same learned inequality, checked at the training tuples and along the closed loop. (b) The closed-loop decrease and the stability condition over the region and in the two boxes around the equilibrium.

What it means

The inequality is satisfied where it was sampled and not where the proof needs it, and the closed loop behaves accordingly, because in the post-fault simulation the learned controller doesn't stabilize the system and the trajectory diverges. The pattern near the equilibrium is the clearest part. Inside the 1% box Φ is positive definite everywhere, so the stability condition itself is fine there, and the inequality still fails at every sampled point, which is what you'd expect if the issue is the u it was trained at rather than the shape of Φ.

The fix I'd try first is to train the inequality where Theorem 2 uses it, either by adding the closed-loop condition at $u = \pi(x)$ to the loss or by drawing u toward the supply-rate minimizer instead of uniformly, and then to repeat exactly this region check. If the networks use activations the verifier supports, the same check becomes a sound CROWN certificate instead of a search, which is the part I'd most like to contribute.

Limits, stated flatly

This is my reproduction and not the authors' controller, so the gap could be in my training, since several hyperparameters aren't in the paper and $\Phi \succ 0$ was only marginally learned in my run, with its smallest eigenvalue near zero because the softplus loss saturates at this scale. Everything here is a dense-sample and gradient-attack search, which can find violations but can't prove their absence, and I couldn't make it a sound

certificate yet because auto_LiRPA 0.7.2 doesn't support the GELU and softplus activations the paper's networks use, while interval arithmetic is too loose on $V(x) - V(x^+)$, a difference of nearly equal values. Current limiting is not modeled. The cleanest way to separate a reproduction issue from a property of the method is to run this same check on the authors' trained controller, and I'm happy to share my code and results with anyone who'd like to look.